

第 15 章

自动测试和测试工具

测试软件是一项艰苦的工作。如果在阅读本书的同时已经进行了一些测试，就会明白执行测试工作需要投入大量的时间和精力。诚然，可以多花一些时间对测试用例进行等价划分，减少执行的数量，但是随之就会冒更多风险，因为减少了测试覆盖范围，选择对某些重要的特性不进行测试。如果测试员需要做更多测试，但是没有时间，怎么办？

答案是采用人们在其他领域和行业用了多年的办法——开发并使用工具，使工作更加轻松和高效。本章要讲的就是这方面的内容。

本章重点包括：

- 为什么测试工具和自动化是必须的
- 可用的简单工具例子
- 工具如何实现测试的自动化
- 如何饲养和照顾“猴子”
- 测试工具和自动化为什么不是灵丹妙药

15.1 工具和自动化的好处

回顾一下软件是如何创建的。在大多数软件开发模式中，软件发布之前要多次重复代码——测试——修复的过程。如果要测试某项特性，也许需要不止一次执行测试，而是重复多次。还要检查确认在前面的测试中发现的软件缺陷修复了，同时又没有引入新的软件缺陷。重复执行测试的过程称为回归测试。

如果一个小型软件项目有数千个测试用例要执行，时间可能只够执行一次测试。多次执行测试是不可能的，更不用说多次测试的单调和枯燥了。通过提供比手工测试更有效的手段，软件测试工具和自动化可以帮助解决这个问题。

工具和自动化的主要属性是：

- **速度。**想一想手工尝试Windows计算器的数千个测试用例要花多少时间。也许大约平均每5秒执行一个测试用例。而自动化可能以10倍、100倍甚至1000倍这样的速度来执行。
- **效率。**当测试员忙于执行测试用例时，他会无暇干别的。如果有一个测试工具减少了执行测试用例的时间，测试员就有更多时间进行测试计划，考虑新的测试用例。
- **准确度和精确度。**测试员尝试几百个测试用例之后，注意力可能会分散，并开始犯错误。而测试工具则会一如既往地每次执行同样的测试，并毫无差错地检查结果。
- **节省资源。**有时要执行一些测试用例基本上是不可能的，比如创建测试条件需要的人数

或设备数目可能就不允许。测试工具可以用来模拟真实的情况，大大减少执行测试需要的物理资源。

- **仿真和模拟。**测试工具常用来代替正常情况下与产品连接的硬件或软件。这些“伪”设备或“伪”程序以选定的方式来驱动或响应软件——除此之外，实现起来可能困难。
- **坚持不懈。**测试工具和自动化永远不会劳累或者半途而废。它们就像电视广告中电动的小兔子——不停地跑啊跑……。

所有这些听起来像重大新闻。测试工具可以代劳所有工作——轻松地使用工具，然后就静观结果。令人遗憾的是，事情并没有那么容易。即使木匠有电锯和钉枪，房屋也不会自动就建好。工具只能使工作更容易、工作结果的质量更高。软件测试工具的使用也是一样。

注意 软件测试工具不能代替软件测试员——它们只能帮助软件测试员更好地工作。

一定要注意使用测试工具不见得总是对的，有时手工测试是不可代替的。目前的任务是了解测试工具能做什么以及怎么做，考虑如何用它们来完成测试任务。本章最后将讲述在着手使用工具测试软件项目之前的一些限制和注意事项。

15.2 测试工具

软件测试员会面对一大堆测试工具。使用工具的类型取决于测试的软件类型，以及是进行黑盒测试还是白盒测试。

测试工具的好处是使用时并不是总需要深入了解工具在怎样做或者做什么。假设正在测试允许一台计算机同时与1百万台计算机连接的网络软件。即使有可能，也很难用1百万个真实的连接进行可以控制的测试。但是，如果有人提供能够模拟那些连接的专用工具，也许可以从1到1百万调整连接的数目，就可以在不搭建实际连接环境的前提下进行测试。测试员不必了解工具是怎样做到的，只要知道它做得到就可以了——这就是黑盒测试。

另一方面，还可以建立工具监视和改变上百万计算机之间原始通信的工具。要有效使用这些工具，测试员需要具备一些白盒技能以及底层协议的知识。

注意 本例体现了两种工具——非入侵式工具和入侵式工具的重大差别。如果工具仅用于监视和检查软件而不对其进行修改，就认为是非入侵式工具。但是如果工具以任何方式修改了程序代码或者控制了操作环境，就属于入侵式工具。由于入侵的程度各有不同，测试员通常设法使用侵入性尽量小的工具，以减少工具影响测试结果的可能性。

下面将讨论测试工具的主要分类及其使用方式。有些例子的工具包含大多数编程语言；而另一些是单独销售的商用工具。然而，有时你会发现自己的软件或硬件很独特，必须自行开发或者让别人来开发符合特定要求的自定义测试工具。虽然如此，它们仍然属于这种分类之一。

15.2.1 查看器和监视器

查看器 (viewer) 或者监视器 (monitor) 测试工具能够看到正常情况下看不到的软件运行的细节。第7章“带上X光眼镜测试软件”讲述了代码覆盖率分析器是如何提供一种方式来查看哪些代码行得以执行、什么函数正在运行、执行测试时所运行的代码分支的。代码覆盖率分析器是查看工具的一个例子。大多数代码覆盖率分析器是入侵式工具，因为它们需要编

译并链接到原程序中才能获得所需信息。

图15-1给出了另一种查看器的例子——通信分析器 (communications analyzer或comm analyzer)。该工具允许查看通过网络或者其他通信电缆传输的原始协议数据。它只是监听线路,提取经过的数据,在另一台计算机上显示。在这个例子中,可以在1号机上输入测试用例,在3号机上确认产生的通信数据是正确的,并在2号机上检测相应的结果。利用该系统还可以观察软件缺陷为什么会产生。通过查看从线上提取的数据,就可以确定问题是出于创建数据(1号机)还是解释数据(2号机)。这种类型的系统对软件是非入侵式的。在网络中,这种监视器被称为嗅探器 (sniffer),因为它像一个电子鼻,从空气中嗅探数据。

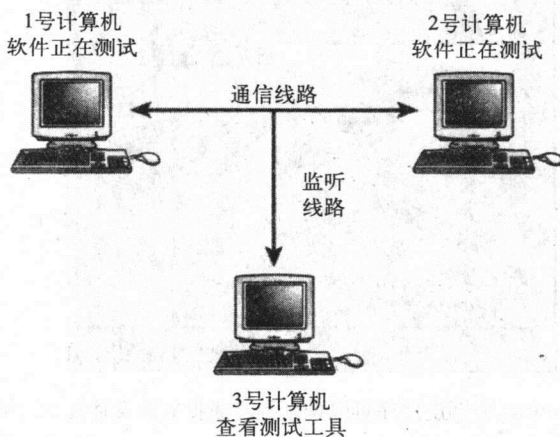


图15-1 通信分析器可以查看两个系统之间传输的原始数据

大多数编译器附带的代码调试器也可以看做查看器,因为它们允许程序员或者白盒测试员查看内部变量值和程序状态。任何能够洞察系统,看到一般用户看不到的数据的工具都可以归类为查看测试工具。

15.2.2 驱动程序

驱动程序是控制和操作被测试软件的工具。最简单的驱动程序例子是批处理文件 (batch file),即依先后顺序执行的程序或命令的一个简单清单。在MS-DOS时代,这是测试员执行测试程序的流行方式。他们会创建包含测试程序名称的批处理文件,启动运行批处理文件,然后返回。在现今的操作系统和编程语言下,执行测试程序有更多复杂的方法。例如,Java和Perl脚本可以取代老的MS-DOS批处理文件,并且Windows任务调度程序(见图15-2)可以在全天24小时的任意时刻执行各种测试程序。

图15-3给出了驱动程序工具的另一个例子。假设正在测试的软件的测试用例需要输入大量数据。随着硬件的一些变更和软件工具的使用,可以用另一台计算机取代原来的键盘和鼠标作为驱动程序来测试。在这台作为驱动程序的计算机上可以编写简单的程序自动产生相应的击键和鼠标移动操作来测试软件。

读者可能会想为什么要这么麻烦使用如此复杂的设置?为什么不是简单地运行一个程序,在第1个系统中执行向被测试软件发送击键信息呢?其中有两个潜在的问题:

- 软件或者操作系统可能不是多任务的,同时运行另一个驱动程序是不可能的。

- 通过从外部计算机发送击键和鼠标移动信息，被测试系统处于非入侵状态。如果测试软件时在同一个系统中执行驱动程序，它就是入侵式的，这种测试情况可能不允许。

在设法驱动被测试的软件时，想一想从外部控制程序的所有可行方法。然后，想办法用自动提供测试输入的方式代替外部控制。

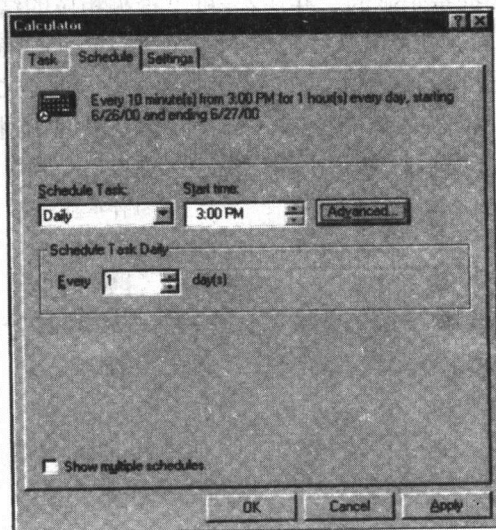


图15-2 Windows任务调度程序可以调度程序或批处理文件在PC上的运行时间

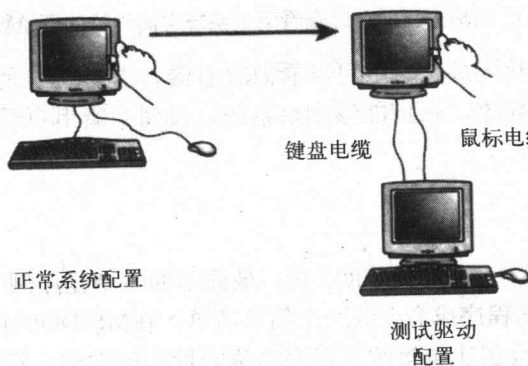


图15-3 一台计算机可以作为驱动程序测试工具取代被测试系统的键盘和鼠标

15.2.3 桩

桩和驱动程序一样，属于第7章所述的白盒测试技术。桩与驱动程序本质上是相反的，桩不控制或者操作被测试软件；相反，它接收或者响应软件发送的数据。图15-4给出的例子有助于澄清这一点。

如果测试向打印机发送数据的软件，一种方法是输入数据并打印，然后查看打印输出结果。这样虽然可行，但是太慢、没有效率、易发生错误。怎样分辨输出是缺了一个像素还是一点点色差？如果换成用运行桩软件的另一台计算机来代替打印机，读取并解释打印数据，就可以非常快捷并且准确地检验测试结果。

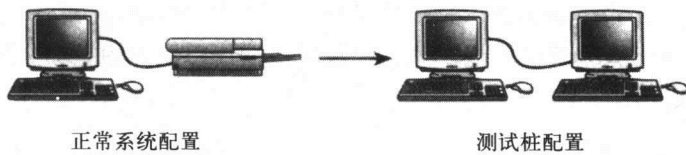


图15-4 一台计算机可以作为桩，代替打印机，以便对测试输出进行更有效的分析

当软件需要与外部设备进行通信时经常要用到桩。一般在开发过程中不能得到这些设备，或者这些设备很少。桩就可以使测试在没有硬件的条件下进行，使测试更加有效。

注意 大家可能听说过仿真器 (emulator) 这个术语。仿真器是在实际使用中用来替代真正设备的设备。把PC机当做一台打印机，像一台真正打印机一样解释打印机编码并响应软件指令，这时PC机就是一台仿真器。仿真器和桩的区别在于桩还给测试程序提供手段来查看和解释发送给它的数据，桩是仿真器的超集。

15.2.4 压力和负载工具

压力 (stress) 和负载 (load) 工具用于向被测试软件增加压力和负载。文字处理程序如果作为系统上唯一运行的应用程序，有足够的内存和磁盘空间，工作状况就可能相当好。但是，如果系统资源不足，就极有可能碰到许多潜在软件缺陷。虽然可以通过复制文件使磁盘占满，运行大量程序耗尽内存等，但是这些方法效率不高，而且不够恰当。为此专门设计的压力工具可以使测试更加容易。

图15-5显示了Microsoft编程语言开发的压力工具软件。其他操作系统和语言也有类似的工具软件。压力程序可以分别设置内存量、磁盘空间大小、文件数量，以及在该机器上运行软件的其他可用资源。

Resource	Remaining
Global	97816.00 KB
User	21 %
GDI	43 %
Disk Space	2047.69 MB
File Handles	120
Wnd32	1943.03 KB
Menu32	1925.17 KB
GDI32	1930.84 KB

图15-5 Microsoft压力工具软件可以设置被测试软件可用的系统资源

把这些值设置为零或者近似为零，会使软件执行不同的代码分支以试图处理这种紧迫限制。理想情况是软件运行不发生崩溃或者数据丢失。它可能运行得很缓慢，或者宣布在内存不足情况下运行，但是无论如何它会正确地运行，或者正常地降级运行。

负载工具和压力工具的相似之处是，它们为软件创造了用其他方式难以创造的环境条件。例如，运行在Web服务器上的商用程序可以通过模拟一定数量的连接和单击次数来增大负载，使其不堪重负。利用它可以检查10 000个模拟用户和每天1百万次单击能否得到处理而不延长响应时间。有了负载工具，就可以方便地设置到该级别、执行测试、观看结果。

15.2.5 干扰注入器和噪声发生器

另一类工具是干扰注入器 (interference injectors) 和噪声发生器 (noise generators)。它们类似于压力和负载工具，但是在行为上更具有随机性。例如，压力工具具有随机更改可用资源的执行模式。某个程序可能在内存充足条件下运行良好，也能应付内存不足状况，但是如果可用内存不断变化，就可能有问题。压力工具的执行模式可以暴露此类软件缺陷。

类似地，可以对如图15-1所示的查看器工具设置进行一点改动，建立如图15-6所示的测试

设置。在这种情况下，查看器由硬件和软件代替，不仅允许查看通信线路上的数据，而且允许更改。这就成为了一个干扰注入器。这样的设置可以模拟所有由数据中断、噪声或者电缆损坏等因素导致的通信错误。

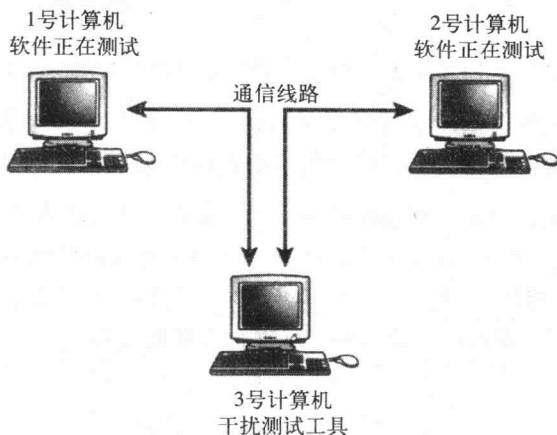


图15-6 挂在通信线路上的干扰注入器可以测试软件能否处理由超声引起的错误情况

决定在哪里和如何使用干扰注入器和噪声发生器时，考虑何种外部因素会影响测试软件，然后设法改变和操纵这些影响因素看软件如何应付。

15.2.6 分析工具

最后一类工具称为分析工具 (analysis tool)，是余下的一组最佳工具。大多数软件测试员利用以下常用工具使日常工作简化。它们不一定像前面的工具一样吸引人。虽然它们常常不受重视，但是它们能够促进测试，节省大量时间。

- 文字处理软件
- 电子表格软件
- 数据库软件
- 文件比较软件
- 抓屏和比较软件
- 调试器
- 二进制——十六进制计算器
- 秒表
- 录像机或者照相机

当然，软件的复杂性和方向性总是在变。要视具体情况来决定最有效的工具是什么，以及如何运用他们。

15.3 软件测试自动化

虽然测试自动化 (test automation) 只是另一类软件测试工具，但是它是值得特别考虑的。到目前为止，所学的软件测试工具虽然确实有效，但是仍然需要手工操作或者监视。假如这些工具结合起来，启动、执行几乎不用人工干预会怎样？它们可以执行测试用例、查找软件

缺陷、分析看到的信息、记录结果。这就是软件测试自动化。

本章以下几节将介绍不同类型的自动化，从最简单的过渡到最复杂的。

15.3.1 宏录制和回放

最基本的测试自动化类型是录制第一次执行测试用例时的键盘和鼠标操作，然后在需要重新执行这些测试时回放一次。如果测试的软件是运行在Windows或Mac上的，录制和回放宏是相当容易的过程。在Mac系统上可以使用Quickeys程序；在Windows上共享程序Macro Magic是上佳之选。由于有许多宏录制和回放程序可用，因此尽可以广泛收罗共享软件，从中找出最符合要求的。

宏录制器和播放器是一种驱动程序工具。如前所述，驱动程序是用于控制和操作被测试软件的工具。利用宏程序，测试员可以这样做——回放录制的宏，重复执行测试软件的操作。

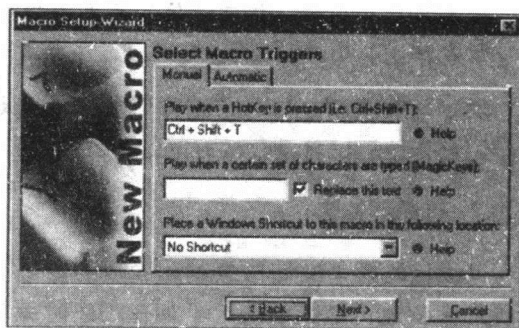


图15-7 Macro Magic设置向导可以配置录制的宏如何触发和回放（图片摘自Iolo Technologies，见www.iolo.com）

图15-7给出了Macro Magic设置向导的一个画面，引领用户进行一步一步配置和捕捉宏所必需的步骤。

Macro Magic设置向导可以设置宏的如下选项：

- **名称。**为宏命名以便以后辨认。即使对于小型软件项目，也可能要编写数百个宏。
- **重复次数。**重复测试是找出软件缺陷的好方法。可以设置宏在运行时重复或者循环的次数。
- **触发条件。**设置宏如何启动，可以按热键（例如 Ctrl + Shift + T）、输入一串字符（例如 run macro 1）、单击快捷方式、当某个窗口显示出来时（例如，一旦计算器启动）或者当系统闲置一段时间之后来启动。
- **捕捉对象。**可以选择只捕获（记录）键盘操作或键盘以及鼠标的移动和单击都记录。
- **回放速度。**宏回放的速度比最初录制时最多慢20%，最多快500%。这对于软件性能可以变化的情形很重要。假如测试的软件变得有点慢，而宏单击的按钮还未出现在屏幕上，结果会怎样？
- **回放位置。**该选项确定鼠标移动和单击位置与某个窗口的位置是绝对的还是相对的。如果测试某个可能在屏幕上改变位置的应用程序，使鼠标移动相对于应用程序是个好主意：否则鼠标单击位置可能不是预期的位置。

现在该实验宏的录制和回放了。寻找和下载一些宏软件，在几个简单程序（例如计算器和记事本）上反复试用，验证想法。要像测试员那样思考！通过实验可以发现虽然宏可以进行一些自动测试，使重新执行测试更加容易和迅速，但是效果并不理想。最大的问题是缺乏验证。宏无法检验软件是否以预期方式进行。宏虽然可以在计算器中输入100 - 99，但是不能测试结果是否为1——测试员仍然要测试。诚然，这是一个问题，但是许多测试员乐于宏消除了所有重复输入和鼠标移动。只观察宏的执行，确认得到预期结果是非常轻松的。

回放速度是宏的另外一个难题。即使可以调整回放的速度，也不见得总能保持宏同步执行。网页可能需要1秒或者10秒来加载。虽然可以根据预计最差情形降低宏的速度，但是它的执行速度就会一直慢，即使软件运行速度变快了也无济于事。此外，如果网页加载意外地用了15秒钟，宏仍然会出现混乱——在错误时刻单击错误的位置。

注意 在使用宏录制器捕捉鼠标移动和单击时要小心。程序并非总是在屏幕上的同一位置启动和出现。把回放位置设置为相对于程序窗口比设置为屏幕的绝对位置更好，但是即使如此，GUI的一丁点变化也会扰乱捕捉的步骤。

除了这些限制之外，录制和回放宏是自动执行简单测试任务的流行方式，同时也是测试员学习如何让测试自动化的好的起点。

15.3.2 可编程的宏

可编程的宏是在简单录制和回放的变化上的一大进步。与其通过录制第一次执行测试时的操作来创建可编程的宏，不如在创建时编写回放系统遵循的简单指令。清单15-1给出了一个非常简单的宏程序（Macro Magic设置向导创建）。此类宏可以通过从菜单中选择独立的操作方式来编程——甚至不必输入命令。

清单15-1 在Windows计算器程序上进行测试的简单宏

```
1: Calculator Test #2
2: <<EXECUTE:C:\WINDOWS\SYSTEM32\Calc.exe----->>
3: <<LOOKFOR:Calculator--SECS:5--->>
4: 123-100=
5: <<PROMPT:The answer should be 23>>
6: <<CLOSE:Calculator>>
```

第1行是标识测试任务的注释行。第2行执行Windows计算器程序Calc.exe。第3行最多用5秒钟等待计算器启动。通过暂停等待标题上显示Calculator的窗口出现。第4行输入123 - 100 =。第5行显示信息提示答案应该为23。第6行关闭窗口，结束测试。

注意，此类可编程的宏与录制的宏相比，具有真正的优势。尽管仍然无法验证测试的结果，但是它可以暂停执行，向测试员提示预期结果，并询问测试是通过还是失败（见图15-8）。

可编程的宏还可以解决录制宏的许多时序问题，不是依靠绝对延时，而是等待特定条件成立才继续执行。在计算器例子中，宏等待程序加载之后继续进行测试——这是比较可靠的方法。

到目前为止，一切顺利。有了可编程的宏，就可以进军自动测试了。现在有一个简单的宏语言，驱动软件的通用命令，以及提示信息的方式。对于许多测试任务，这已经够用了，按这种方式自动测试可以节省大量时间。

然而，在进行复杂测试时还遗漏了两个重要之处。首先，可编程的宏限于直接执行命令行——只能循环和重复。在常规编程语言中可以见到的变量和决策语句不能使用，而且没有

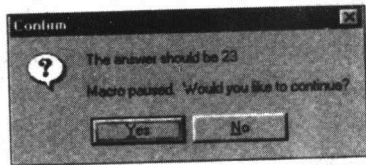


图15-8 简单编程的宏不能验证测试结果，但是可以提示测试员进行确认（图片摘自Iolo-Technologies，见www.iolo.com）

自动检查测试结果的能力。因此，需要向全面的自动测试工具发展。

15.3.3 完全可编程的自动测试工具

如果具有成熟编程语言的能力，加上驱动被测试软件的宏命令，以及进行验证的能力，结果会怎样？就可以得到从根本上具备查找软件缺陷能力的工具！图15-9给出了此类工具的一个例子。

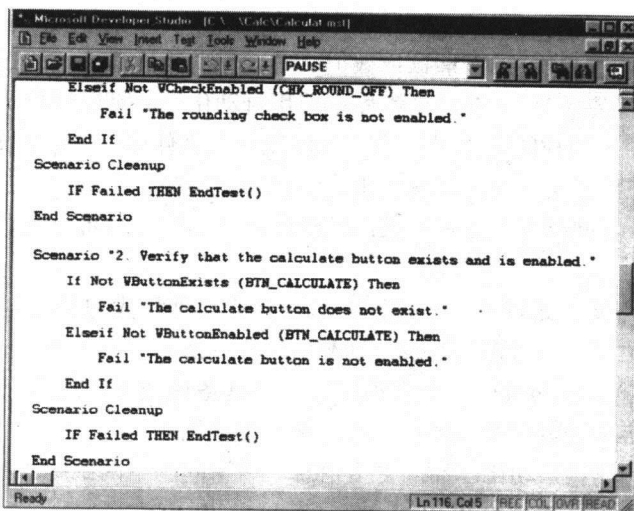


图15-9 可视化测试，最初由Microsoft公司开发，现在由IBM支持，是在一个软件包中提供编程环境、宏命令和验证能力的工具例子

像可视化测试程序这样的自动测试工具为软件测试员创建强大测试提供了手段。许多工具基于BASIC编程语言，即使非程序员也可以轻松地编写测试代码。

如果想输入10 000遍字符串“Hello World!”，就可以编写如下所示的几行代码来实现：

```
FOR i=1 TO 10000
PLAY "Hello World!"
NEXT I
```

如果要在640 × 480的屏幕上把鼠标指针从左上角移到右下角，然后双击，就可以编写如下语句实现：

```
PLAY "{MOVETO 0,0}"
PLAY "{MOVETO 640,480}"
PLAY "{DBLCLICK}"
```

测试语言还可以提供比单纯单击某个屏幕区域或者发送单个击键操作更好的控制特性。例如，要单击OK按钮，可以使用以下命令：

```
wButtonClick ("OK")
```

不必知道OK按钮在屏幕上的位置。测试软件可以找到它并单击——和用户操作一样。类似地，还有菜单、复选框、单选按钮和列表框等命令。此类命令为编写测试用例提供了极大的便利，使其可读性和可靠性更好。这些自动化工具具有的最重要特点是进行验证的能力，实际上就是检查软件是否以预期方式运行。实现这一点有以下几种方式：

- **屏幕捕获。**首次执行自动测试时，可以在肯定正确的关键点捕捉并保存屏幕图像。在以后进行测试时，自动化工具可以利用保存的屏幕画面与当前屏幕画面进行比较。如果两者不同，就说明出现意外，自动化工具会把它标记为软件缺陷。

注意，使用屏幕捕获进行测试有相当大量的工作要进行维护。即使一个像素点的变化都会导致屏幕比较失败。除非软件的用户接口不变化，否则在每次测试时进行手工捕获和比较，那就违反了自动测试的目的了。

- **控件值。**比屏幕捕获更进一步，可以检查软件窗口中各种控件的值。如果测试计算器程序，自动化工具就会从显示区域读取数值，与预期结果进行比较。还可以判断是否单击了按钮或者选中了复选框。在测试程序中使用自动化工具可以轻松实现这一点。
- **文件和其他输出。**同样的，如果软件把数据保存在文件中——例如文字处理程序——自动化工具就能在建立文件之后将其读出，与已知正确的文件比较。该技术同样适用于通过调制解调器或者网络发送数据的被测试软件。自动化工具可以配置为读回数据与预期数据进行比较。

和屏幕捕获一样，文件比较也有问题。如果文件或文件格式包含日期、计数器或其他变化值，文件比较就会失败。这时需要修改自动测试工具，忽略这些差别。

验证是自动测试要克服的最后一个大难题。一旦具备该能力，就可以对绝大多数测试用例进行验证，以更加容易或者完全自动化的方式执行测试用例。

关于市面上流行的测试自动化产品的更多信息，请访问以下网站：

- Software Development Technologies at www.sdtcorp.com
- Mercury at www.mercury.com
- Segue Software at www.segue.com

这些软件包对于个人使用可能有点昂贵，因为它们主要针对的是企业的测试小组。但是，如果对获得它们的一些使用经验有兴趣，可以和相应的公司联系，索取评估版；假如你是学生，可以申请学生打折优惠。大多数软件工具公司都会提供帮助，使你喜欢它们的产品，并最终推荐给其他人。

15.4 随机测试：猴子和大猩猩

到目前为止，我们学习的测试自动化工具和技术主要是使软件测试员的工作更加轻松和富有成效。其设计目标是帮助测试员执行测试用例，或者在理想状态下，自动执行测试用例，而不必管它。

出于该目的，使用工具和自动化有助于找出软件缺陷；在工具忙着进行回归测试时，软件测试员就有更多时间计划新的测试，设计新奇有趣的用例。然而，另一类自动测试不是为帮助执行或者自动执行测试用例而设计的，其目标是模拟用户可能的操作。此类自动化工具称为测试猴子（test monkey）。

测试猴子的说法来源于一个想法，如果让一百万只猴子在一百万只键盘上敲一百万年，从统计的角度讲，它们最终就可能写出莎士比亚话剧 *Adventures of Curious George*（好奇乔治历险记）等巨著。胡乱敲键可能无意间碰到正确的字母组合，这样猴子瞬间变得才华横溢，如图15-10所示。

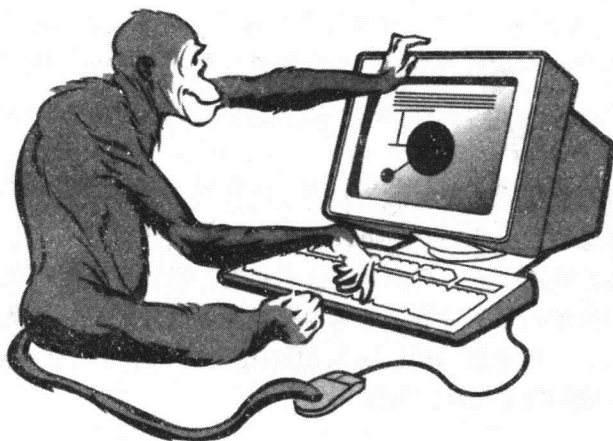


图15-10 只要不停电，偶尔能够得到香蕉，测试猴子就会永远测试下去

当软件公开发布后，可能会有成千上万的人使用。除非尽最大努力设计测试用例，查找缺陷，否则有些软件缺陷就会漏掉，而被这些用户发现。在产品发布之前，如果用模拟这些用户可能的操作的方式来补充测试用例，结果会怎样？这就有可能发现以其他测试方式会漏掉的软件缺陷。这正是测试猴子做的工作。

注意 利用测试猴子模拟客户使用软件的方式没有一点暗示或映射计算机用户和猿猴有关的意思。

15.4.1 笨拙的猴子

最简单最直接的测试猴子是笨拙的猴子。笨拙的猴子一点也不了解被测试软件：只是随机地单击鼠标或者敲击按键。清单15-2给出了随机单击鼠标和按键10 000次的可视化测试代码例子。

清单15-2 只需几行代码即可创建笨拙的猴子

```
1: RANDOMIZE TIMER
2: FOR i=1 TO 10000
3: PLAY "{CLICK "+STR$(INT(RND*640))+", "+STR$(INT(RND*480))+"}"
4: PLAY CHR$(RND*256)
5: NEXT i
```

第1行初始化随机数。第2行从1到10 000开始循环。第3行从屏幕的0, 0位置到639 479 (VGA 分辨率) 位置随机选择一个屏幕单击。第4行从0到255随机挑选一个字符输入。

PC机上运行的软件不知道上面这个程序与真人操作之间的区别——除了程序的运行速度非常快之外。该程序在正常速度的PC机上运行只需几秒钟。可以想像一下如果通宵运行该程序，将会得到多少随机输入！

记住，笨拙的猴子绝对不会进行验证。它只是单击鼠标和敲击按键，直至两件事之一发生——或者完成循环，或者软件、操作系统崩溃。如果被测试的软件崩溃，笨拙的猴子并不会知道，它还会继续地单击鼠标和敲击按键。

注意 如果不相信笨拙的猴子可能会发现严重缺陷,就试试在平时喜爱的计算机游戏或者多媒体程序上执行笨拙的猴子。这些程序极有可能支持不了几个小时就会崩溃。

只依靠单击鼠标和敲击按键来发现软件缺陷也许没有什么意义,但是这样做有如下两个原因:

- 如果有足够的时间和精力,就像猴子写莎士比亚名著一样,随机输入最终可能打出程序员和测试员没有想到的奇特序列。也许猴子输入了一些数据马上又删除了,或者在该输入短字符串的地方输入了长字符串。谁会想到呢?但是猴子能找到。
- 不停重复和使用笨拙的猴子可能会暴露内存泄漏等软件缺陷,这类缺陷要在正常使用软件数小时或者数天之后才能出现。如果用过随着使用时间长变得越来越不稳定的软件,就见到了利用笨拙的猴子发现的问题。

15.4.2 半聪明的猴子

笨拙的猴子可能特别有效,它们易于编写,可能会发现导致崩溃的严重缺陷。然而,它们缺乏一些使其更加有效的重要特性。增加这些特性来提高猴子的智商,使其成为半聪明的猴子。

假设测试猴子运行了几个小时,在软件崩溃之前记录了成千上万种随机输入。软件测试员可能知道其中有问题,但是无法向程序员说清楚如何重现它。可以用同样的随机序列重新运行猴子,但是如果花了几个小时又使软件失败了,就只是在浪费宝贵的时间。解决方法是在测试猴子中增加日志,将测试猴子所有的操作记录到一个文件中。当测试猴子发现软件缺陷时,只需查看日志文件,找出在失败之前做了些什么。

技巧 另外一个解决跟踪测试猴子行为的办法是用摄像机录下屏幕上的内容。当发现软件出现故障时,就倒回去重新看看。

编写猴子程序,只在被测试的软件上运行也是一个好想法。如果猴子程序在屏幕上到处随机单击,就会(最终会)单击到退出命令终止程序。由于猴子不知道程序关闭了,因此它还会继续运行。想一想如果猴子在计算机屏幕上到处单击,结果会怎么样——哦!大多数可编程自动化工具提供了一种方式,使测试总是针对某个特定应用程序,或者在应用程序关闭了停止工作。

使猴子变聪明的另一个优点是崩溃辨认能力。如果启动猴子程序,通夜运行,但测试员一走出房间它就发现了一个软件缺陷,就会浪费很多宝贵的测试时间。如果为猴子编写新功能,使其能识别发生了崩溃,并重新启动计算机,然后重新开始测试,每晚就可能多找出几个软件缺陷。

15.4.3 聪明的猴子

半聪明的猴子再继续进化就成为聪明猴子。这样的猴子从它不够聪明的兄弟那里获得随机测试的结果,并增加了对环境的认知能力。它不单单随机乱敲键盘——而是有目的地敲。

真正聪明的猴子知道:

- 它在哪里
- 在那里能干什么

- 它能到哪里
- 它曾经在哪里
- 所见到的是否正确

这个清单是否有点熟悉？是的。聪明猴子会阅读软件的状态转换图——第5章“带上眼罩测试软件”所讲述的那种图。如果猴子可以看懂描述软件的全部状态信息，它就可以像用户那样试用软件，只是使用速度更快，而且可以验证遇到的情况。

聪明猴子测试Windows 计算器（见图15-11）能知道有哪些按钮可以单击，存在哪些菜单项，在哪里输入数字。如果它单击Help菜单的About Calculator选项，就知道唯一的退出方法是单击OK按钮或者Close按钮。它不会在屏幕上到处随机单击，最终意外命中目标。

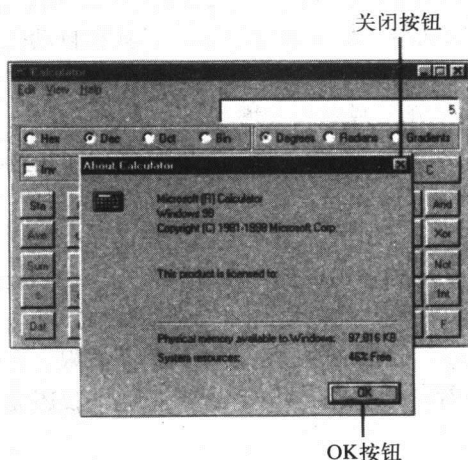


图15-11 聪明猴子应该知道如何关闭About Calculator对话框

聪明猴子也不仅限于查找崩溃缺陷。它同时时查看数据、检查操作结果、找出其与预期结果的差别。如果在测试用例中编程设计聪明猴子，它可以随机执行、查找软件缺陷、记录结果。太酷了！

图15-12显示了一个叫Koko的聪明猴子，名字来源于会说符号语言的猿猴。要想编程设计Koko，就得提供通过定义所有状态来描述软件的状态表、该状态中可以执行的操作，以及决定操作执行结果是否正确的声明。

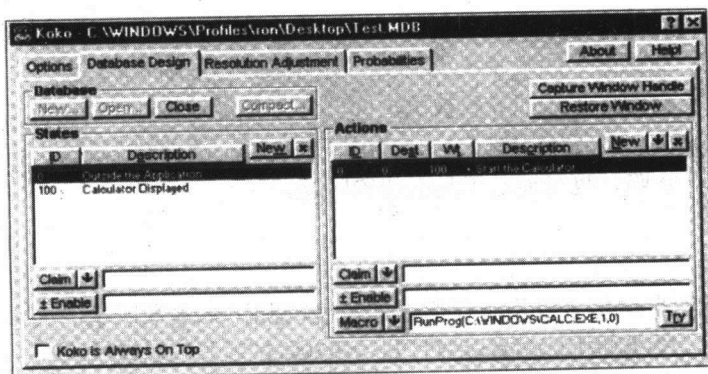


图15-12 聪明猴子Koko可以编写成知道自己在哪里，可以干什么

当Koko运行时，它驱动软件到已知状态，根据模拟实际情形的权衡结果，随机选择一个操作，执行该操作，然后检查结果。如果该操作导致软件改变状态，Koko就会知道，并使用适用于新状态的一组新操作。

有了这样的系统，就可以模拟软件的实际使用，把数千个小时的使用时间压缩为几个小时。聪明猴子是真正的寻找缺陷机器！

注意 Koko是作者开发的一个适度聪明的猴子，并不用于商业应用。

15.5 使用测试工具和自动化的实质

在满怀喜悦地准备开始在测试中使用测试工具和自动化之前，需要阅读本节并记在心里。测试自动化不是万能的。如果正确规划和执行的话，工具和自动化可以使测试效率大大提高并且能发现其他方式不能发现的缺陷。然而，如果自动化和工具步入歧途，会导致无数的自动化测试的努力被放弃，并且使项目成本大大增加。

在开始使用本章所述的技术之前，应该对以下这些重要的问题加以考虑：

- 软件变更。产品说明书从未修复过。后期增加了新特性。产品名称在最后一刻被更改。如果录制了数千个宏执行全部测试，而在产品发布的前一周，软件改为启动时多显示一屏，结果会怎样？所有录制的宏都将运行失败，因为它们不知道多出一屏。这时，需要编写的自动化程序具备灵活性，在需要时能够方便快捷地改变。
- 人眼和直觉是不可替代的。编出来的聪明猴子的聪明有限。它们只能执行人交给它们的测试。它们永远不可能看到什么说：“呀，这很有趣。我还要做更多检查”——至少目前不会。
- 验证难以实现。如果测试用户界面，验证测试结果最简单、最显然的方法是捕捉和比较屏幕画面。但是，屏幕画面是大型文件，而且这些屏幕画面在产品开发过程中不断变化。要保证测试工具只检查需要的画面，而且能够在产品开发过程中高效处理变化。
- 容易过分依赖自动化。永远不要因为执行了全部自动化测试没有发现软件缺陷，就认为没有软件缺陷要找了。软件缺陷仍然有，这是杀虫剂怪现象。
- 不要花费太多时间使用达不到测试软件目的的测试工具和自动化。开始编写宏和聪明猴子虽然轻松愉快，但这不是测试。这些工具有助于提高效率，但是需要在软件运行时使用它们，并进行实际的测试，才能找出软件缺陷。
- 编写宏、开发工具和编制猴子都属于开发工作。软件测试员应该遵守要求程序员遵守的标准和规范。仅仅因为你是测试员并不意味着可以破坏规则。
- 某些工具是入侵式的，可能导致被测试的软件不正常地失败。如果使用工具发现了一个软件缺陷，要设法在不使用该工具的情况下重现这个软件缺陷。结论可能是一个简单的可重现的软件缺陷，也可能是工具引起的问题。

15.6 小结

软件测试工具和自动化可以用于任何软件类型的测试。本章所示的大多数示例是关于图形用户界面的测试的，但是同样的技术可以运用到编译器、网络和Web服务器的测试中。好好想一想需要执行的测试任务，如何利用软件使其更加容易和快速实现——这正是自动化的

领域。

人们有时徒手干，有时使用苍蝇拍，有时（也许不太雅观）使用锤子。清楚何时使用工具和使用哪一种工具是软件测试员的重要技能。创建、使用测试工具和测试自动化是有趣的工作。看到计算机自己在运行、光标来回闪烁、字符自动输入的感觉很酷。当你躺在床上，或者出去喝一杯时，自动化工具吱吱地工作，查找软件缺陷，你会感到相当地满意。

15.7 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. 说出使用软件测试工具和自动化的一些好处。
2. 在决定使用软件测试工具和自动化时，要考虑哪些缺点或者注意事项？
3. 工具和自动化之间有何差别？
4. 查看工具和注入工具有何异同？
5. **判断是非**：入侵式工具是最佳类型，因为其操作与测试的软件最贴近。
6. 最简单但很有效的测试自动化类型是什么？
7. 说出可以增加到问题6中所述的测试自动化中，使其更有效的一些特性。
8. 聪明猴子比笨和笨拙的猴子有什么优点？